MIT 6.8610 Quantitative Natural Language Processing

Spring 2026

Final Project Proposal

Joshua Gallego Pereira Winter Tao Gabriel Vargas

March 2026

1 Motivation

Prompt-based adaptation is a parameter-efficient alternative to full fine-tuning [Li and Liang, 2021, Lester et al., 2021, Liu et al., 2023], but it often trades off expressivity and interpretability. Continuous prompts are flexible but difficult to interpret [Khashabi et al., 2022], while discrete prompts are easier to inspect but constrained by natural language [Shin et al., 2020, Mishra et al., 2022, Liu et al., 2023]. Linear prompt ensembling partially bridges this gap by expressing a continuous prompt as a weighted combination of discrete prompt embeddings [Passigan et al., 2023], though it remains limited to linear interpolation over a fixed prompt bank. In this project, we ask whether a lightweight modern language model can learn effective reasoning strategies through linear combinations of embedded discrete prompts. We focus on mathematical reasoning, where correctness and strategy choice are easy to verify and analyze. Our approach uses a bank of discrete reasoning prompts and learns how to combine them, either with supervised training or with reinforcement learning, allowing us to compare accuracy and interpretability within the same linear prompt-mixing framework.

2 Proposed Research

We propose a reinforcement-learning approach to learning linear combinations of embedded discrete prompts. Let f_θ denote a lightweight language model with embedding dimension d and prompt length m. In our main experiments, f_θ will be Qwen2.5-7B-Instruct. Rather than fully fine-tuning the backbone, we keep it frozen or lightly adapted and train only a small prompt-mixing module. We begin with a fixed prompt bank $\{s_k\}_{k=1}^{K}$, where each prompt encodes a reusable reasoning strategy, and let $p_k \in \mathbb{R}^{m\times d}$ denote the embedding of prompt s_k. Given an input problem x, we compute a pooled representation of the input embeddings,

$$\bar{e}(x) = \frac{1}{\sum_i m_i}\left(\sum_i m_i E(x)_i\right),$$

where $E(x)_i$ is the embedding of token i and m_i is the attention mask. A learned prompt-mixing network h maps this representation to a distribution over prompts,

$$\alpha(x) = \mathrm{softmax}(h(\bar{e}(x))) \in \Delta^{K-1},$$

and we construct the final prompt as a linear combination of prompt embeddings,

$$P(x) = \sum_{k=1}^{K} \alpha_k(x)\, p_k.$$

The resulting model prediction is

$$\hat{y} = f_\theta([P(x); E(x)]).$$

We study two training regimes for this same linear prompt-mixing mechanism. In the supervised setting, the prompt weights are learned directly from labeled examples. In the reinforcement-learning setting, learning the linear prompt combination is treated as a policy-learning problem: the state is the input problem x, the action is the weight vector $\alpha(x)$, and the outcome is the answer generated by the backbone under the resulting prompt. For mathematical reasoning tasks with canonical final answers, we use a binary correctness reward,

$$r(x) = \begin{cases} 1 & \text{if the predicted answer is correct,} \\ 0 & \text{otherwise.} \end{cases}$$

We optimize the prompt-mixing network with a REINFORCE-style policy-gradient objective

$$\nabla_\theta J(\theta) = \mathbb{E}_{\alpha\sim\pi_\theta(\cdot|x)}\big[r(x,\alpha)\,\nabla_\theta \log \pi_\theta(\alpha \mid x)\big],$$

where $\pi_\theta(\alpha \mid x)$ is the policy over prompt weights induced by h. Intuitively, this objective increases the probability of linear prompt combinations that lead to correct answers. We intentionally restrict the method to linear combinations of embedded discrete prompts so that any difference between the supervised and reinforcement-learning settings can be attributed to the training objective rather than to added representational capacity. This setup is interpretable because the prompt weights reveal which reasoning strategies are emphasized, and efficient because only the prompt-mixing module is trained.

3 Related Work

Prompt tuning [Lester et al., 2021] and prefix tuning [Li and Liang, 2021] show that frozen language models can be adapted with a small number of learned prompt parameters, although the resulting prompts are difficult to interpret [Khashabi et al., 2022]. Discrete prompt optimization improves readability but remains constrained by natural-language prompt design [Shin et al., 2020, Mishra et al., 2022, Liu et al., 2023]. Linear prompt ensembling [Passigan et al., 2023] provides a useful middle ground by expressing a continuous prompt as a convex combination of embeddings from a fixed bank of discrete prompts, yielding a more interpretable decomposition than unconstrained continuous prompting. Our work builds directly on this formulation by comparing two ways of learning the combination weights: supervised training and reinforcement learning. More broadly, the project is motivated by the idea that answer supervision alone may not be sufficient to learn effective reasoning behavior, especially on math tasks where multiple strategies are possible. Rather than applying reinforcement learning to all model parameters, we apply it only to linear prompt mixing, which keeps the project tractable while preserving interpretability.

4 Evaluation

We evaluate the method along two axes: mathematical reasoning accuracy and interpretability of the learned policy. Our primary benchmark is GSM8K [Cobbe et al., 2021], a dataset of grade-school math word problems requiring multi-step reasoning. Given a problem x, the model generates a final answer $\hat{y}$ under the mixed prompt, and we use normalized exact-match accuracy as the primary metric,

$$\text{Acc} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}[\hat{y}_i = y_i].$$

In our first experiment, we compare three systems: a fixed discrete prompting baseline, a supervised linear combination of embedded discrete prompts, and a reinforcement-learning-based linear combination of embedded discrete prompts. This tests whether reward-driven linear prompt mixing improves over both static prompting and supervised linear prompt mixing. Further, we inspect the learned weight distributions $\alpha(x)$ for emergent legible patterns. By plotting weights by problem type and difficulty, we assess whether the RL policy develops interpretable mixing strategies.

In our second experiment, we evaluate performance in an adversarial prompt-bank setting that includes both helpful and intentionally misleading mathematical strategies, such as false equalities, invalid operations, division-by-zero errors, and contradictory heuristics. This tests whether the learned policy can identify and suppress harmful reasoning strategies rather than relying on them.

In our third experiment, we compare all three methods against Qwen2.5-Math-7B-Instruct [Yang et al., 2024] as a same-size math-specialized benchmark to estimate how far fixed prompting, supervised linear prompt mixing, and reinforcement-learning-based linear prompt mixing can move a general-purpose Qwen2.5-7B-Instruct model [Qwen et al., 2025] toward the performance of a math-specialized counterpart.

5 Feasibility and Implementation Plan

The project will be implemented in PyTorch using HuggingFace Transformers [Paszke et al., 2019, Wolf et al., 2020]. To keep the system feasible, we will use Qwen2.5-7B-Instruct as the main backbone, kept frozen or adapted only minimally with a small LoRA configuration if needed. The main trainable component is the prompt-mixing network h, making the setup substantially cheaper than full fine-tuning or full-model reinforcement learning. We will begin by validating the pipeline on a small subset of GSM8K and then scale to the full benchmark once the reinforcement-learning objective behaves stably. Experiments will be run on an OCI `VM.GPU.A10.2` instance with 2× NVIDIA A10 GPUs, which should be sufficient for a frozen or lightly adapted 7B backbone with a small prompt-mixing module.

References

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv*, 2021. `https://arxiv.org/abs/2110.14168`.

Daniel Khashabi, Xinxi Lyu, Sewon Min, Lianhui Qin, Kyle Richardson, Sean Welleck, Hannaneh Hajishirzi, Tushar Khot, Ashish Sabharwal, Sameer Singh, and Yejin Choi. Prompt waywardness: The curious case of discretized interpretation of continuous prompts. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3631–3643, 2022.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021.

Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9):1–35, 2023.

Swaroop Mishra, Daniel Khashabi, Chitta Baral, Yejin Choi, and Hannaneh Hajishirzi. Reframing instructional prompts to gptk's language. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 589–612, 2022.

Pascal Passigan, Kidus Yohannes, and Joshua Pereira. Continuous prompt generation from linear combination of discrete prompt embeddings. *arXiv preprint arXiv:2312.10323*, 2023.

Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, 2019.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv*, 2025. URL `https://arxiv.org/abs/2412.15115`.

Taylor Shin, Yasaman Razeghi, Robert L. Logan IV, Eric Wallace, and Sameer Singh. Autoprompt: Eliciting knowledge from language models with automatically generated prompts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020.

Thomas Wolf, Lysandre Debut, Victor Sanh, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement. *arXiv*, 2024. URL `https://arxiv.org/abs/2409.12122`.